1. Write a C program to simulate SJF scheduling algorithm non preemptive.

```c
#include<stdio.h>

struct process{
    int pid;
    int at;
    int bt;
    int ct;
    int wt;
    int tat;
    int rt;
    int completed;
};

void sjf_np(struct process p[],int n){
    int time=0,done=0,min,index;
    float total_tat=0,total_wt=0,total_rt=0;
    while(done!=n){
        min=999;
        index=-1;

        for(int i=0;i<n;i++){
            if(p[i].at<=time && p[i].completed==0){
                if(p[i].bt<min){
                    min=p[i].bt;
                    index=i;
                }
            }
        }

        if(index==-1){
            time++;
            continue;
        }

        p[index].rt=time-p[index].at;
        time+=p[index].bt;

        p[index].ct=time;
        p[index].tat=p[index].ct-p[index].at;
        p[index].wt=p[index].tat-p[index].bt;

        p[index].completed=1;
        done++;
    }

    for(int i=0;i<n;i++){
        total_wt+=p[i].wt;
```

```
        total_tat+=p[i].tat;
        total_rt+=p[i].rt;
    }

    printf("PID\tAT\tBT\tCT\tWT\tTAT\tRT\n");
    for(int i=0;i<n;i++){
        printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",p[i].pid,p[i].at,p[i].bt,p[i].ct
,p[i].wt,p[i].tat,p[i].rt);
    }

    printf("Average waiting time=%.2f\n",total_wt/n);
    printf("Average turn around time=%.2f\n",total_tat/n);
    printf("Average response time=%.2f\n",total_rt/n);
}

int main(){

    int n;
    printf("enter number of processes:");
    scanf("%d",&n);

    struct process p[n];

    for(int i=0;i<n;i++){
        printf("\nProcess %d\n",i+1);
        p[i].pid=i+1;
        printf("Arrival Time:");
        scanf("%d",&p[i].at);
        printf("Burst Time:");
        scanf("%d",&p[i].bt);
        p[i].completed=0;
    }

    sjf_np(p,n);

    return 0;
}
```

Output:

```
Process 1
Arrival Time:0
Burst Time:3

Process 2
Arrival Time:2
Burst Time:2

Process 3
Arrival Time:3
Burst Time:5

Process 4
Arrival Time:4
Burst Time:4

Process 5
Arrival Time:6
Burst Time:1
PID     AT      BT      CT      WT      TAT     RT
1       0       3       3       0       3       0
2       2       2       5       1       3       1
3       3       5       15      7       12      7
4       4       4       9       1       5       1
5       6       1       10      3       4       3
Average waiting time=2.40
Average turn around time=5.40
Average response time=2.40
```

2. Write a C program to simulate SJF scheduling algorithm preemptive.

```c
#include<stdio.h>

struct process{
    int pid;
    int at;
    int bt;
    int rt_bt;
    int ct;
    int wt;
    int tat;
    int rt;
};

void sjf_pa(struct process p[],int n){
    int time=0,completed=0,min,index;
    float total_tat=0,total_wt=0,total_rt=0;
    while(completed!=n){
        min=999;
        index=-1;

        for(int i=0;i<n;i++){
            if(p[i].at<=time && p[i].rt_bt>0){
                if(p[i].rt_bt<min){
                    min=p[i].rt_bt;
                    index=i;
                }
                else if(p[i].rt_bt==min){
                    if(p[i].at<p[index].at){
                        index=i;
                    }
                }
            }
        }

        if(index==-1){
            time++;
            continue;
        }

        if(p[index].rt==-1){
            p[index].rt=time-p[index].at;
        }
        p[index].rt_bt--;
        time++;

        if(p[index].rt_bt==0){
            p[index].ct=time;
```

```
            p[index].tat=p[index].ct-p[index].at;
            p[index].wt=p[index].tat-p[index].bt;
            completed++;
        }
    }

    for(int i=0;i<n;i++){
        total_wt+=p[i].wt;
        total_tat+=p[i].tat;
        total_rt+=p[i].rt;
    }

    printf("PID\tAT\tBT\tCT\tWT\tTAT\tRT\n");
    for(int i=0;i<n;i++){
        printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",p[i].pid,p[i].at,p[i].bt,p[i].ct
,p[i].wt,p[i].tat,p[i].rt);
    }

    printf("Average waiting time=%.2f\n",total_wt/n);
    printf("Average turn around time=%.2f\n",total_tat/n);
    printf("Average response time=%.2f\n",total_rt/n);
}

int main(){

    int n;
    printf("enter number of processes:");
    scanf("%d",&n);

    struct process p[n];

    for(int i=0;i<n;i++){
        printf("\nProcess %d\n",i+1);
        p[i].pid=i+1;
        printf("Arrival Time:");
        scanf("%d",&p[i].at);
        printf("Burst Time:");
        scanf("%d",&p[i].bt);
        p[i].rt_bt=p[i].bt;
        p[i].rt=-1;
    }

    sjf_pa(p,n);

    return 0;
}
```

Output:

```
Process 1
Arrival Time:0
Burst Time:3

Process 2
Arrival Time:2
Burst Time:2

Process 3
Arrival Time:3
Burst Time:5

Process 4
Arrival Time:4
Burst Time:4

Process 5
Arrival Time:6
Burst Time:1
PID     AT      BT      CT      WT      TAT     RT
1       0       3       3       0       3       0
2       2       2       5       1       3       1
3       3       5       15      7       12      7
4       4       4       10      2       6       1
5       6       1       7       0       1       0
Average waiting time=2.00
Average turn around time=5.00
Average response time=1.80
```

3. Write a C program to simulate priority scheduling algorithm non preemptive.

```c
#include<stdio.h>

struct process{
    int pid;
    int at;
    int bt;
    int pr;
    int ct;
    int wt;
    int tat;
    int rt;
    int completed;
};

void priority_np(struct process p[],int n){
    int time=0,done=0,high,index;
    float total_tat=0,total_wt=0,total_rt=0;

    while(done!=n){
        high=999;
        index=-1;

        for(int i=0;i<n;i++){
            if(p[i].at<=time && p[i].completed==0){
                if(p[i].pr < high){
                    high=p[i].pr;
                    index=i;
                }
            }
        }

        if(index==-1){
            time++;
            continue;
        }

        p[index].rt=time-p[index].at;
        time+=p[index].bt;

        p[index].ct=time;
        p[index].tat=p[index].ct-p[index].at;
        p[index].wt=p[index].tat-p[index].bt;

        p[index].completed=1;
        done++;
    }
```

```c
    for(int i=0;i<n;i++){
        total_wt+=p[i].wt;
        total_tat+=p[i].tat;
        total_rt+=p[i].rt;
    }

    printf("PID\tAT\tBT\tPR\tCT\tWT\tTAT\tRT\n");
    for(int i=0;i<n;i++){
        printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i].pid,p[i].at,p[i].bt,p[i].pr,p[i].ct,p[i].wt,p[i].tat,p[i].rt);
    }

    printf("Average waiting time=%.2f\n",total_wt/n);
    printf("Average turn around time=%.2f\n",total_tat/n);
    printf("Average response time=%.2f\n",total_rt/n);
}

int main(){

    int n;
    printf("enter number of processes:");
    scanf("%d",&n);

    struct process p[n];

    for(int i=0;i<n;i++){
        printf("\nProcess %d\n",i+1);

        p[i].pid=i+1;

        printf("Arrival Time:");
        scanf("%d",&p[i].at);

        printf("Burst Time:");
        scanf("%d",&p[i].bt);

        printf("Priority:");
        scanf("%d",&p[i].pr);

        p[i].completed=0;
    }

    priority_np(p,n);

    return 0;
}
```

Output:

```
Process 1
Arrival Time:0
Burst Time:3
Priority:5

Process 2
Arrival Time:2
Burst Time:2
Priority:3

Process 3
Arrival Time:3
Burst Time:5
Priority:2

Process 4
Arrival Time:4
Burst Time:4
Priority:4

Process 5
Arrival Time:6
Burst Time:1
Priority:1
PID     AT      BT      PR      CT      WT      TAT     RT
1       0       3       5       3       0       3       0
2       2       2       3       11      7       9       7
3       3       5       2       8       0       5       0
4       4       4       4       15      7       11      7
5       6       1       1       9       2       3       2
Average waiting time=3.20
Average turn around time=6.20
Average response time=3.20
```

4. Write a C program to simulate priority scheduling algorithm preemptive.

```c
#include<stdio.h>

struct process{
    int pid;
    int at;
    int bt;
    int pr;
    int rt_bt;
    int ct;
    int wt;
    int tat;
    int rt;
};

void priority_pa(struct process p[],int n){
    int time=0,completed=0,high,index;
    float total_tat=0,total_wt=0,total_rt=0;

    while(completed!=n){
        high=999;
        index=-1;

        for(int i=0;i<n;i++){
            if(p[i].at<=time && p[i].rt_bt>0){
                if(p[i].pr < high){
                    high=p[i].pr;
                    index=i;
                }
                else if(p[i].pr == high){
                    if(p[i].at < p[index].at){
                        index=i;
                    }
                }
            }
        }

        if(index==-1){
            time++;
            continue;
        }

        if(p[index].rt==-1){
            p[index].rt=time-p[index].at;
        }

        p[index].rt_bt--;
        time++;
```

```
        if(p[index].rt_bt==0){
            p[index].ct=time;
            p[index].tat=p[index].ct-p[index].at;
            p[index].wt=p[index].tat-p[index].bt;

            completed++;
        }
    }

    for(int i=0;i<n;i++){
        total_wt+=p[i].wt;
        total_tat+=p[i].tat;
        total_rt+=p[i].rt;
    }

    printf("PID\tAT\tBT\tPR\tCT\tWT\tTAT\tRT\n");

    for(int i=0;i<n;i++){
        printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i].pid,p[i].at,p[i].bt,p[i].pr,p[i].ct,p[i].wt,p[i].tat,p[i].rt);
    }

    printf("Average waiting time=%.2f\n",total_wt/n);
    printf("Average turn around time=%.2f\n",total_tat/n);
    printf("Average response time=%.2f\n",total_rt/n);
}

int main(){

    int n;
    printf("enter number of processes:");
    scanf("%d",&n);

    struct process p[n];

    for(int i=0;i<n;i++){
        printf("\nProcess %d\n",i+1);

        p[i].pid=i+1;

        printf("Arrival Time:");
        scanf("%d",&p[i].at);

        printf("Burst Time:");
        scanf("%d",&p[i].bt);

        printf("Priority:");
```

```
        scanf("%d",&p[i].pr);

        p[i].rt_bt=p[i].bt;
        p[i].rt=-1;
    }

    priority_pa(p,n);

    return 0;
}
```

Output:

```
Process 1
Arrival Time:0
Burst Time:3
Priority:5

Process 2
Arrival Time:2
Burst Time:2
Priority:3

Process 3
Arrival Time:3
Burst Time:5
Priority:2

Process 4
Arrival Time:4
Burst Time:4
Priority:4

Process 5
Arrival Time:6
Burst Time:1
Priority:1
PID     AT      BT      PR      CT      WT      TAT     RT
1       0       3       5       15      12      15      0
2       2       2       3       10      6       8       0
3       3       5       2       9       1       6       0
4       4       4       4       14      6       10      6
5       6       1       1       7       0       1       0
Average waiting time=5.00
Average turn around time=8.00
Average response time=1.20
```